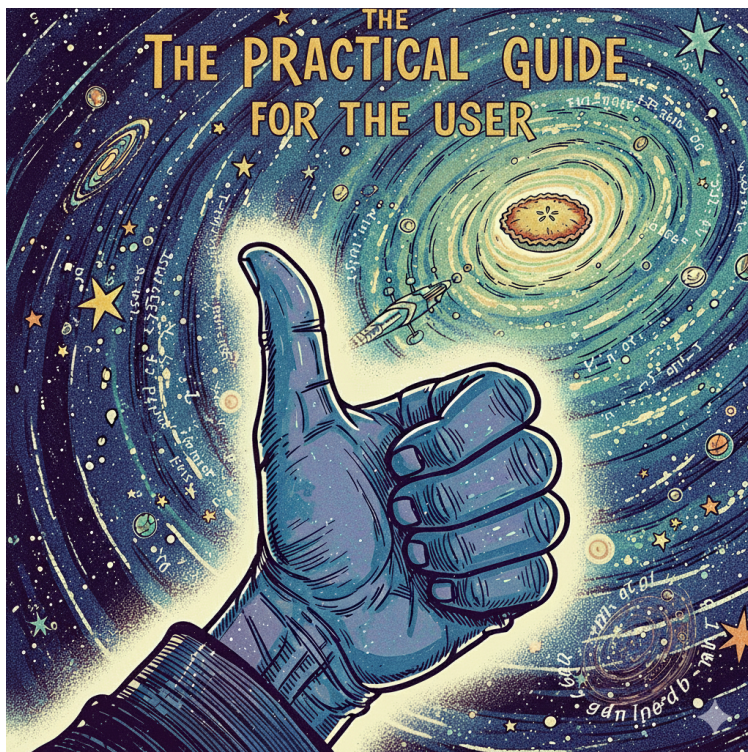


PrimePy: Number Theory Library

Practical User Guide

Jonatan Beiruty

24.7.2025



Contents

1 Introduction

This user guide provides a practical overview of how to use the `PrimePy` library. Rather than focusing on the underlying mathematical theory or algorithmic design, this document is dedicated to demonstrating how to work with the library from a programmer's perspective.

Through concise and focused examples, users will learn how to import and interact with the various components of the library, including core number theory utilities, algebraic structures, and cryptographic primitives. Each section includes annotated code snippets and explanations to help both beginners and advanced users integrate `PrimePy` into their own projects.

For the theoretical background and detailed explanation of the algorithms implemented, please refer to the accompanying document.

2 Algebra

This section introduces the core abstract algebraic structures supported by the `PrimePie` library. We begin with a brief overview of each structure and then provide concrete examples of their implementations within the library.

The main structures currently supported are:

1. **Groups**
2. **Rings**

2.1 Additive Group Modulo n ($\mathbb{Z}/n\mathbb{Z}$)

The Python API exposes a thin, Pythonic wrapper over the C++ implementation. Instantiate the additive group modulo n and use the unified `power` method, which dispatches by argument shapes:

- `power(int base, int exponent) -> int`
- `power(Iterable[int] bases, int exponent) -> list[int]`
- `power(Iterable[int] bases, Iterable[int] exponents) -> list[int]`

Example (basic usage).

```
from primepie.algebra.algebraic_structures import AdditiveModGroup
```

```
G = AdditiveModGroup(7)           # the additive group ( $\mathbb{Z}/7\mathbb{Z}$ , +)
```

```
print(G.identity())               # 0
print(G.inverse(3))               # 4   (since  $3 + 4 \equiv 0 \pmod{7}$ )
print(G.operate(3, 6))           # 2   ( $3 + 6 \equiv 2 \pmod{7}$ )
```

```
# unified power (additive repeated operation)
print(G.power(2, 3))              # 6   ( $2 + 2 + 2 \equiv 6 \pmod{7}$ )
print(G.power([1,2,3], 2))       # [2, 4, 6]
print(G.power([1,2,3], [0,1,2])) # [0, 2, 6]
```

Notes.

- The group law is addition mod n , so `power(b, k)` means adding `b` to itself `k` times (with negative exponents handled via the additive inverse).
- All results are reduced into $[0, n - 1]$.

2.2 Ring of Integers Modulo n ($\mathbb{Z}/n\mathbb{Z}$)

The ring wrapper exposes both:

- The **additive group interface** (`identity`, `inverse`, `operate`, and `additive power`).
- The **ring primitives** (`zero`, `one`, `add`, `neg`, `mul`, `is_equal`, `contains`).
- A separate multiplicative power method, called `mpower`, which performs repeated multiplication under modulo n .

Example (basic usage).

```
from primepie.algebra.algebraic_structures import IntegersModRing
```

```
R = IntegersModRing(7)           # the ring Z/7Z

print(R.zero(), R.one())         # 0 1
print(R.add(5, 6))               # 4 (11 4)
print(R.neg(3))                  # 4 (-3 4)
print(R.mul(3, 5))               # 1 (15 1)
print(R.is_equal(10, 3))         # True

# additive interface inherited from Group
print(R.identity())              # 0
print(R.inverse(3))              # 4
print(R.operate(3, 6))           # 2
print(R.power(2, 3))             # 6 (2+2+2 6)

# multiplicative power via mpower
print(R.mpower(3, 0))            # 1
print(R.mpower(3, 4))            # 4 (3^4 = 81 4)
print(R.mpower([1,2,3,6], 3))    # [1, 1, 6, 6]
print(R.mpower([2,3,4,5],[0,1,2,3])) # [1, 3, 2, 6]
```

Notes.

- **Additive vs. Multiplicative Power.**
 - `power` uses the additive group structure: $a^k = a + \dots + a$ (k times).
 - `mpower` uses the multiplicative structure: $a^k = a \cdot \dots \cdot a$ (k times).
- **Negative Exponents.** Additive powers accept negative exponents (using inverses). Multiplicative powers do not (unless the structure is a field, to be added in future versions).

- **Zero Ring** ($n = 1$). In $\mathbb{Z}/1\mathbb{Z}$ we have $0 = 1$, so both `zero()` and `one()` return 0, and every product and sum is 0.